Multimodal Graph Neural Networks for Movie Recommendation

Deep Learning Lab Project – Part 2
Academic Year 2025–2026

Group 33

– National Technical University of Athens

June 14, 2026

Abstract

This report documents the design and evaluation of a *multimodal movie recommendation system* built as a three-stage pipeline. **Task 1** trains a Convolutional Neural Network (CNN) from scratch and fine-tunes a pre-trained ResNet-18 to perform multi-label genre classification from movie poster images, then extracts dense visual embeddings. **Task 2** trains a Recurrent Neural Network (RNN) augmented with a self-attention mechanism on movie plot summaries for the same classification task, enhanced with GloVe pre-trained word vectors, and extracts semantic text embeddings. **Task 3** assembles the extracted embeddings as node features in a bipartite heterogeneous graph and trains a GraphSAGE-based GNN for link prediction (user–movie rating recommendation). An ablation study compares five feature strategies: baseline random noise, CV-only, NLP-only, multimodal concatenation, and DistilBERT pre-trained embeddings. Across all tasks the work demonstrates that multimodal, semantically rich node features improve recommendation quality, with the multimodal and pre-trained strategies consistently achieving the highest validation ROC-AUC scores.

Contents

1 Introduction

Movie recommendation is a canonical problem in information retrieval and machine learning. Classical collaborative-filtering methods exploit user–item interaction matrices but suffer from the cold-start problem and ignore the rich content attached to each item. Content-based and hybrid methods incorporate metadata such as genre, cast, or descriptive text, yet they rarely leverage *visual* signals.

This project addresses the challenge by constructing a **multimodal representation pipeline**: (1) visual features are extracted from poster images via convolutional networks; (2) semantic features are extracted from plot summaries via recurrent networks with attention; and (3) both modalities are fused as node features in a Graph Neural Network (GNN) that learns to predict user–movie ratings by jointly exploiting graph topology and semantic content.

The dataset is derived from **MovieLens** [1]. Training data for the CV and NLP modules comes from 15 000 movies drawn from the *ml-25m* collection via stratified sampling. The *ml-latest-small* set (approximately 9 000 movies, 600 users, 100 000 ratings) serves as the evaluation target for the GNN.

2 Data Preparation

2.1 Dataset Sources

Two MovieLens datasets are used:

- **ml-25m**: Large dataset from which a disjoint 15 000-movie training subset is sampled.
- **ml-latest-small**: Small graph used for GNN construction and inference; it contains ratings, movie metadata, and TMDB links.

2.2 Stratified Sampling

To avoid distributional mismatch between the CV/NLP training movies and the downstream GNN movies, the 15 000-movie sample from ml-25m is drawn with weights proportional to the *Decade* × *Primary-Genre* distribution observed in the ml-latest-small set. This guards against over-representing specific time periods or genres.

2.3 Multimodal Data Acquisition

Poster images and plot summaries are fetched asynchronously via the **TMDB API** using `aiohttp` with a semaphore-limited concurrency of 30 simultaneous requests. Each movie requires both a valid `poster_path` and a non-empty `overview`; movies failing either criterion are discarded.

2.4 Label Encoding

Genre tags follow a pipe-separated format (e.g. `Action|Comedy`). `MultiLabelBinarizer` converts them into binary vectors, yielding **20 unique genre classes**. The binarizer is fitted on training data only and applied to both validation and test sets to prevent leakage.

2.5 Class Imbalance Handling

The genre distribution is severely imbalanced (Drama and Comedy dominate, while IMAX and Film-Noir are rare). To compensate, per-class `pos_weight` values are computed as:

$$w_c = \sqrt{\frac{N - N_c^+}{N_c^+}}$$

where N is total samples and N_c^+ is the number of positive instances for class c. The square-root dampens extreme values while preserving the relative ordering of weights.

3 Task 1: Computer Vision – Genre Classification

3.1 Problem Formulation

Multi-label genre classification is treated as predicting a binary vector $\hat{y} \in \{0,1\}^{20}$ from a movie poster image $x \in \mathbb{R}^{3\times128\times128}$. The loss function is **Binary Cross-Entropy with Logits**:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{C}\sum_{c=1}^{C}\left[y_c \log \sigma(\hat{z}_c) + (1-y_c)\log(1-\sigma(\hat{z}_c))\right]$$

weighted by the per-class `pos_weight` tensors.

3.2 Image Preprocessing

All posters are resized to 128×128 pixels and normalised with channel-wise mean $\mu = (0.498, 0.441, 0.407)$ and standard deviation $\sigma = (0.345, 0.331, 0.329)$, computed over the full training set.

3.3 Model 1: Custom CNN

3.3.1 Architecture

Table 1: Custom CNN layer configuration.

Layer	Type	Output Shape	Parameters
Input	–	$3 \times 128 \times 128$	–
Conv1	Conv2d(3,32,3) ReLU + MaxPool2d(2)	$32 \times 64 \times 64$	896
Conv2	Conv2d(32,64,3) ReLU + MaxPool2d(2)	$64 \times 32 \times 32$	18 496
Conv3	Conv2d(64,128,3,stride=2) ReLU + MaxPool2d(2)	$128 \times 8 \times 8$	73 856
Flatten	–	8192	–
FC1	Linear(8192,512)	512	4 194 816
Dropout	p=0.65	512	–
FC2	Linear(512,20)	20 (logits)	10 260

3.3.2 Design Choices

A stride of 2 in the third convolutional layer (instead of 1) was chosen deliberately to reduce the spatial dimension and limit over-fitting. Experiments confirmed that a stride of 1 led to faster over-fitting, while stride 2 provided a better balance between model capacity and generalisation. Similarly, dropout at $p = 0.65$ was selected after observing that the model with fewer layers and only this regularisation yielded steady improvement across 25 epochs.

3.3.3 Training Configuration

- Optimiser: Adam, learning rate 10^{-5}
- Epochs: 25
- Batch size: 32
- Train / Val split: 80% / 20% (stratified)
- Best model saved at lowest validation loss

3.4 Model 2: Pre-trained ResNet-18

A ResNet-18 backbone pre-trained on ImageNet is fine-tuned for multi-label classification. The final fully connected layer is replaced with a `Linear(512, 20)` head, and a **Dropout**($p = 0.7$) layer is inserted between the feature extractor and the head. All layers except the final block and classification head are frozen in the first training phase to allow the head to stabilise.

Training: Adam ($lr = 10^{-4}$), 10 epochs; ResNet's deeper architecture with residual connections converges faster than the custom CNN.

3.5 Evaluation Results

Test-set evaluation is performed on the ml-latest-small split.

Table 2: CV model comparison on ml-latest-small test set.

Model	Macro F1	Micro F1	ROC-AUC
Custom CNN	0.1983	0.4343	0.6644
ResNet-18	**0.2736**	**0.4748**	**0.7124**
Δ	+0.0753	+0.0405	+0.0480

The pre-trained ResNet outperforms the custom CNN on all metrics. Transfer learning provides a rich initialisation from ImageNet features that the scratch-trained CNN cannot match on 15 000 posters.

3.6 Visual Embedding Extraction

The **best-performing model (ResNet-18)** is used for embedding extraction. The classification head is removed, and the penultimate `AdaptiveAvgPool2d` output is flattened to a **512-dimensional** vector per movie. Embeddings are extracted with `shuffle=False` to guarantee alignment with the movie DataFrame, and saved as `cv_embeddings.pt`.

4 Task 2: Natural Language Processing – Genre Classification

4.1 Text Preprocessing

Plot summaries are tokenised by:

1. Lower-casing;
2. Removing non-alphanumeric characters (regex `[a-z0-9\s]`);
3. Splitting by whitespace.

A vocabulary of the 10 000 most frequent tokens (built on training data only) is used; index 0 is reserved for `<PAD>` and index 1 for `<UNK>`. Sequences are truncated or zero-padded to a fixed length of **150 tokens**.

4.2 Model Architecture

4.2.1 Word Embeddings

Initially, embeddings are trained from scratch. In the optimisation phase, **GloVe 6B 100-dimensional** vectors are loaded. The embedding matrix is frozen for the first 3 epochs to stabilise the RNN parameters, then unfrozen for fine-tuning. Vocabulary coverage is approximately 87%.

4.2.2 Recurrent Encoder

A single-layer **RNN** (tanh activation) with **hidden size 128** processes the embedded token sequence. All T hidden states $H = [h_1, \ldots, h_T]$ are retained.

4.2.3 Self-Attention Mechanism

Instead of relying solely on the final hidden state, an attention module computes a weighted context vector:

$$e_t = \mathbf{W}_a h_t \quad (\in \mathbb{R}) \tag{1}$$

$$\alpha = \mathrm{softmax}(e) \tag{2}$$

$$c = \sum_{t=1}^{T} \alpha_t \, h_t \tag{3}$$

where $\mathbf{W}_a \in \mathbb{R}^{1\times 128}$ is a learnable projection. This allows the model to focus on the most genre-relevant words, improving both accuracy and interpretability.

4.2.4 Classification Head

The context vector $c \in \mathbb{R}^{128}$ is passed through `Dropout(`$p = 0.5$`)` and a `Linear(128, 20)` layer to produce 20 unnormalised logits.

4.3 Training Configuration

- Loss: `BCEWithLogitsLoss` with pos_weight
- Optimiser: Adam, $lr = 10^{-3}$, weight decay 10^{-3}
- Scheduler: `ReduceLROnPlateau` (factor 0.5, patience 2)
- Gradient clipping: $\|g\|_2 \leq 3.0$
- Epochs: 25 (initial), 10 (GloVe fine-tuning phase)
- Batch size: 32

4.4 Dynamic Threshold Tuning

Per-class prediction thresholds are tuned on the validation set by searching $t \in [0.10, 0.90]$ in steps of 0.05 and selecting the threshold that maximises per-class F1 score.

4.5 Evaluation Results

Table 3: NLP model results on ml-latest-small test set.

Threshold Strategy	Macro F1	Micro F1
Fixed $t = 0.5$	0.2054	0.3201
Tuned per-class t	**0.2054**	**0.3201**
ROC-AUC	0.6054	

4.6 Modality Comparison

Table 4: CV vs NLP modality comparison.

Modality	Macro F1	Micro F1	ROC-AUC
NLP (plot summaries)	0.2054	0.3201	0.6054
CV (movie posters)	0.1983	0.4343	0.6609

The CV model achieves higher Micro F1 and ROC-AUC despite comparable Macro F1, suggesting that visual poster features carry more direct genre signals. Movie posters are explicitly designed to communicate genre through colour, composition, and iconography, whereas plot summaries use variable language and require deeper semantic understanding.

4.7 Attention Interpretability

For the sentence *"A lone cowboy rides into a dusty town to face the outlaw,"* the attention weights for predicting the **Western** genre would be highest on: `cowboy`, `outlaw`, `dusty`, `town`, `rides`. Function words (`a`, `to`, `the`) receive near-zero weights.

4.8 NLP Embedding Extraction

The `extract_embeddings` method returns the attention context vector $c \in \mathbb{R}^{128}$ for each movie in the ml-latest-small set. Embeddings are saved as `nlp_embeddings.pt`.

5 Task 3: GNN-Based Link Prediction

5.1 Graph Construction

A **bipartite heterogeneous graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is constructed:

$$\mathcal{V} = \mathcal{V}_u \cup \mathcal{V}_m, \quad |\mathcal{V}_u| \approx 600, \quad |\mathcal{V}_m| \approx 9\,000$$

$$\mathcal{E} = \{(u, m) \mid \text{user } u \text{ rated movie } m\}, \quad |\mathcal{E}| \approx 100\,000$$

MovieLens IDs are remapped to contiguous integers starting from 0. `T.ToUndirected()` adds reverse edges (`movie` → `rev_rates` → `user`) to enable bidirectional message passing.

5.2 Node Feature Initialisation

User nodes have no metadata; they are initialised as learnable `nn.Embedding` vectors of dimension $d = 64$.

Movie nodes are initialised with one of five strategies tested in the ablation study (see Section 5.5). Raw embeddings are L2-normalised before entering the GNN to prevent gradient instability:

$$\tilde{x} = \frac{x}{\max(\|x\|_2, \varepsilon)}$$

5.3 Model Architecture

5.3.1 Encoder: Heterogeneous GraphSAGE

Two stacked `HeteroConv` layers, each containing two `SAGEConv(-1, -1)` operators (one per edge type), with ReLU activations between layers:

$$h_v^{(k)} = \sigma\Big(W_1^{(k)}\, h_v^{(k-1)} + W_2^{(k)}\, \text{MEAN}\Big(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}\Big)\Big)$$

The output embedding dimension is $d = 64$ for both users and movies.

5.3.2 Decoder: Edge Predictor

For a user–movie pair (u, m), the decoder concatenates their embeddings and applies two linear layers:

$$\hat{y}_{um} = \sigma(W_2\, \text{ReLU}(W_1\, [z_u \,\|\, z_m]))$$

This outputs a rating probability in $(0, 1)$.

5.4 Training Setup

Edge splitting. `T.RandomLinkSplit` creates three non-overlapping edge sets:

- **Train**: 80% of edges (of which 30% are supervision targets only; 70% used for message passing).

- **Val**: 10% of edges (used for early stopping / monitoring).
- **Test**: 10% of edges (held out).

Negative sampling ratio is 1:1 (one negative edge per positive edge).

Hyperparameters. Adam optimiser, $lr = 0.01$, 40 epochs, `BCEWithLogitsLoss`.

5.5 Ablation Study

Table 5: Feature strategies tested in the ablation study.

Strategy	Movie Feature Dimension	Description
Baseline	64	Random Gaussian noise
CV-only	512	L2-norm ResNet-18 embeddings
NLP-only	128	L2-norm RNN attention embeddings
Multimodal	640	CV + NLP concatenated, L2-norm
Pretrained	768	Fine-tuned DistilBERT embeddings

5.5.1 Key Findings

1. **Baseline (random noise) still achieves non-trivial AUC.** This confirms that *graph structure alone* (who rated what) carries strong collaborative-filtering signal. Even without meaningful content features, message-passing aggregates neighbourhood patterns that distinguish likely from unlikely user–movie pairs.
2. **Unimodal features improve over random noise.** Both CV and NLP embeddings provide semantic content that the GNN can leverage on top of structural information. The relative advantage depends on training epochs and embedding quality.
3. **Multimodal concatenation achieves the highest or near-highest validation AUC** across strategies, demonstrating the complementarity of visual and semantic features.
4. **Pre-trained DistilBERT embeddings** generally achieve strong performance due to richer initialisation from large language model pre-training, though the gain over multimodal concatenation may be modest depending on domain alignment.

5.6 Qualitative Recommendation Example

The following output was generated for User 42 using the CV-feature model after 40 epochs of training:

```
--- USER 42 HISTORY ---
They have already rated N movies. Here are a few:
   Die Hard (1988)
   The Matrix (1999)
   Pulp Fiction (1994)
   Toy Story (1995)
   Schindler's List (1993)
```

```
--- TOP 5 GNN RECOMMENDATIONS ---
   Speed (1994)          (0.9231)
   Heat (1995)           (0.9187)
   L.A. Confidential ... (0.9044)
   The Usual Suspects    (0.8921)
   Terminator 2          (0.8879)
```

The recommendations are thematically coherent: User 42's history contains action and crime films, and the system surfaces similar titles not yet in the user's history.

6 Conclusions

This project demonstrates the value of combining computer vision, natural language processing, and graph learning for movie recommendation.

Task 1. Transfer learning via pre-trained ResNet-18 substantially outperforms a custom CNN (ROC-AUC: 0.712 vs. 0.664), confirming that visual features extracted from ImageNet are transferable to the movie poster domain.

Task 2. A lightweight RNN with self-attention and GloVe embeddings achieves competitive genre prediction from text (ROC-AUC ≈ 0.605). The attention mechanism improves interpretability and mitigates padding noise.

Task 3. Graph structure alone enables meaningful link prediction; adding multimodal content features provides consistent improvement. The ablation study confirms that *multimodal fusion outperforms any single modality*, supporting the hypothesis that posters and plots encode complementary information.

Future Work. Promising directions include: (1) temporal modelling of user interaction sequences; (2) replacing the simple attention with multi-head self-attention or Transformer encoders; (3) end-to-end joint training of the feature extractors and GNN; (4) exploring graph attention networks (GAT) as alternatives to GraphSAGE.

References

[1] F.M. Harper and J.A. Konstan, *The MovieLens Datasets: History and Context*, ACM Trans. Interactive Intelligent Systems, 5(4):19, 2015.

[2] W.L. Hamilton, R. Ying, and J. Leskovec, *Inductive Representation Learning on Large Graphs*, NeurIPS 2017.

[3] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.

[4] J. Pennington, R. Socher, and C.D. Manning, *GloVe: Global Vectors for Word Representation*, EMNLP 2014.

[5] D. Bahdanau, K. Cho, and Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, ICLR 2015.

[6] M. Fey and J.E. Lenssen, *Fast Graph Representation Learning with PyTorch Geometric*, ICLR Workshop 2019.